

MASTER PRODUCT BOOK

GoReadCode

Plataforma Inteligente de Compreensao, Engenharia Reversa
e Inteligencia de Software

Understand Any Code. Anywhere.

STATUS	Em Planejamento
TIPO	SaaS + Desktop + IDE Extensions
CATEGORIA	Engenharia de Software IA Educacao
DATA	June 2026

SUMÁRIO

PARTE I — PRODUTO

- Cap. 1 — Visão e Missão
- Cap. 2 — Mercado e Público-Alvo
- Cap. 3 — Problema e Solução
- Cap. 4 — Posicionamento e Diferenciais
- Cap. 5 — Onde o GoReadCode Pode Ser Utilizado
- Cap. 6 — Fontes de Entrada Suportadas
- Cap. 7 — Funcionalidades
- Cap. 8 — Requisitos Funcionais e Não Funcionais
- Cap. 9 — Fluxo do Usuário
- Cap. 10 — Roadmap
- Cap. 11 — Métricas de Sucesso
- Cap. 12 — Monetização
- Cap. 13 — Riscos
- Cap. 14 — Constituição do Produto

PARTE II — ARQUITETURA DE SOFTWARE

- Cap. A — Princípios Arquiteturais
- Cap. B — Modelo de Domínio (DDD)
- Cap. C — Stack Tecnológica
- Cap. D — Arquitetura Modular
- Cap. E — Engine de Análise e Indexação
- Cap. F — Eventos de Domínio
- Cap. G — Infraestrutura e DevOps

PARTE III — INTELIGÊNCIA ARTIFICIAL

- Cap. A — Arquitetura da IA
- Cap. B — Agentes Especializados
- Cap. C — RAG e Banco Vetorial
- Cap. D — Memória e Contexto
- Cap. E — Prompt Engine e Reasoning

PARTE IV — VISUALIZAÇÃO INTELIGENTE

- Cap. A — Intelligent Visualization Engine
- Cap. B — Preview e Sandbox
- Cap. C — Diagramas e Fluxos

PARTE V — ESPECIFICAÇÃO TÉCNICA

- Cap. A — Banco de Dados
- Cap. B — APIs REST e GraphQL
- Cap. C — Segurança

PARTE VI — UX/UI

- Cap. A — Identidade Visual

Cap. B — Telas e Componentes

Cap. C — Acessibilidade e Responsividade

PARTE VII — MANUAIS E ANEXOS

Anexo A — Comparativo Competitivo

Anexo B — Glossário

Anexo C — Visão de Longo Prazo (5–10 anos)

PARTE I

PRODUTO

Visão, Mercado, Funcionalidades, Roadmap e Estratégia

CAPÍTULO 1

Visão e Missão

O GoReadCode é uma plataforma inteligente de compreensão, análise, documentação, aprendizado e depuração de código-fonte. Seu principal diferencial é que ela nunca tenta alterar um código antes de compreendê-lo completamente.

1.1 O que é o GoReadCode?

O GoReadCode é uma plataforma de Code Intelligence que combina análise estática, engenharia reversa automatizada e Inteligência Artificial para transformar qualquer base de código em conhecimento estruturado e acessível. Ao receber um projeto, a plataforma executa uma análise profunda para construir conhecimento completo sobre arquitetura, regras de negócio, dependências, qualidade, segurança, testes e performance — e somente então atua como assistente de desenvolvimento, documentação, debugging e ensino.

1.2 Missão

Democratizar a compreensão de software complexo através de Inteligência Artificial, transformando qualquer código em conhecimento acessível.

1.3 Visão

Ser a principal plataforma mundial de leitura inteligente de código-fonte, tornando qualquer projeto compreensível independentemente da linguagem, framework ou arquitetura.

1.4 Tagline

Understand Any Code. Anywhere.

CAPÍTULO 2

Mercado e Público-Alvo

O GoReadCode atende diferentes perfis profissionais, do estudante ao arquiteto sênior.

2.1 Personas

Persona	Necessidade Principal
Lucas — Dev Júnior	Quer aprender programação utilizando projetos reais como material didático.
Ana — Tech Lead	Precisa compreender rapidamente bases de código desconhecidas ao assumir novos projetos.
Pedro — Arquiteto	Audita sistemas corporativos antes de propor mudanças arquiteturais.
Mariana — QA	Precisa compreender funcionalidades e fluxos antes de criar planos de teste.
João — Professor	Usa o GoReadCode durante aulas para demonstrar código real a alunos.
Bruno — DevOps	Entende pipelines, scripts de infraestrutura e arquitetura de deploy.
Clara — Segurança	Realiza auditorias de vulnerabilidades em sistemas legados.
Diego — Estudante	Aprende desenvolvimento de software através da leitura guiada de projetos open source.

2.2 Segmentos de Mercado

- Desenvolvedores individuais (freemium / Pro)
- Equipes de desenvolvimento (plano Teams)
- Empresas corporativas (plano Enterprise)
- Instituições de ensino (plano Educação)
- Comunidade open source

CAPÍTULO 3

Problema e Solução

Compreender software legado ou desconhecido é um dos maiores custos invisíveis da engenharia de software.

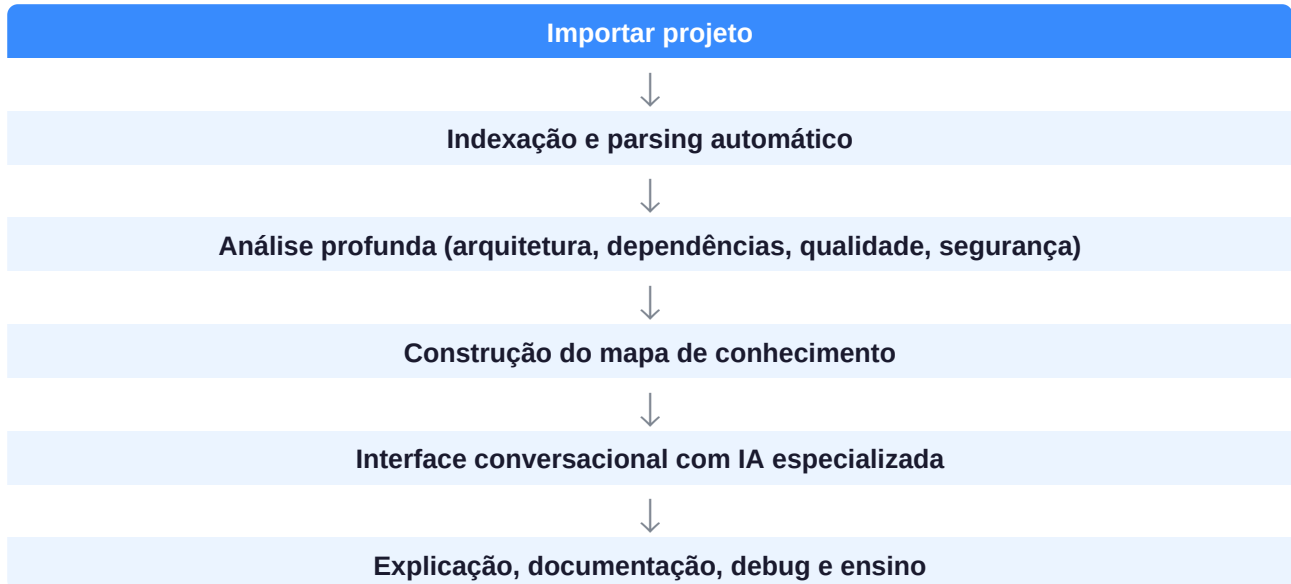
3.1 O Problema

Projetos de médio e grande porte possuem milhares de arquivos, centenas de dependências, documentação desatualizada, regras de negócio implícitas e arquiteturas de difícil compreensão. Um desenvolvedor gasta em média dias — às vezes semanas — tentando entender um sistema antes de conseguir modificá-lo com segurança. Esse tempo tem custo direto para times e empresas.

- Onboarding técnico lento e custoso
- Documentação ausente ou desatualizada
- Regras de negócio implícitas no código
- Medo de alterar sistemas legados sem entendê-los
- Dificuldade em ensinar programação com projetos reais
- Auditorias de segurança manuais e imprecisas

3.2 A Solução

O GoReadCode reduz o tempo de compreensão de dias para minutos. A plataforma analisa automaticamente o projeto completo e constrói um mapa de conhecimento que permite ao desenvolvedor entender qualquer parte do sistema — do panorama arquitetural até a explicação linha a linha — com precisão e rastreabilidade.



CAPÍTULO 4

Posicionamento e Diferenciais Competitivos

4.1 Diferenciais

Diferencial	Descrição
Compreensão antes da ação	O GoReadCode nunca modifica código sem antes construir contexto completo sobre o sistema.
Explicações em múltiplos níveis	Do panorama arquitetural (C4, diagramas) até a análise linha a linha.
Foco em ensino	Transforma bases de código complexas em material didático interativo.
Análise unificada	Arquitetura, qualidade, segurança, testes, performance e documentação em uma única plataforma.
Rastreabilidade total	Toda conclusão é ligada à sua evidência no código-fonte.
Extensibilidade	Preparada para integração com IDEs, repositórios Git e agentes especializados por terceiros.
Multi-LLM e Multi-Fonte	Suporte a diferentes provedores de IA e diferentes origens de código.

4.2 Comparativo com o Mercado

Principais concorrentes e ferramentas adjacentes

- GitHub Copilot — foco em geração de código, não em compreensão
- Cursor / Windsurf — IDEs com IA, sem análise arquitetural profunda
- Sourcegraph — busca e navegação de código sem análise semântica ampla
- SonarQube — qualidade e segurança, sem explicação didática
- CodeScene — análise comportamental, sem modo professor ou documentação automática
- ChatGPT / Claude — análise de trechos isolados, sem indexação do projeto completo

O GoReadCode ocupa um espaço único: plataforma completa de Code Intelligence com foco em compreensão, ensino, auditoria e documentação — não em geração automática de código.

CAPÍTULO 5

Onde o GoReadCode Pode Ser Utilizado

A plataforma foi projetada para acompanhar o usuário em qualquer contexto.

- Em casa — para estudos, projetos pessoais e aprendizado autodidata
- Em escolas, universidades, cursos técnicos e bootcamps como ferramenta de ensino
- Em ambientes corporativos — para onboarding, auditoria, manutenção e evolução de sistemas
- Em equipes de desenvolvimento — para revisão de código e compartilhamento de conhecimento
- Em pesquisas acadêmicas e científicas
- Em projetos Open Source
- Em startups em fase de crescimento acelerado
- Em grandes empresas com sistemas legados complexos

CAPÍTULO 6

Fontes de Entrada Suportadas

O GoReadCode analisa software a partir de diferentes origens, oferecendo flexibilidade para qualquer cenário.

6.1 Repositórios Git

- GitHub (público e privado mediante autenticação)
- GitLab
- Bitbucket
- Azure DevOps
- Gitea
- Servidores Git corporativos e self-hosted

6.2 Código-Fonte Direto

- Upload de arquivos individuais
- Upload de pastas compactadas (.zip, .tar, .rar)
- Projetos completos via arrastar e soltar (Drag & Drop)
- Cole diretamente na interface (modo paste)

6.3 Análise por URL de Aplicação

O usuário poderá informar apenas o endereço de uma aplicação web. O sistema realizará análise inteligente das informações públicas acessíveis, identificando:

- Frameworks de frontend detectáveis
- Bibliotecas JavaScript expostas
- Headers HTTP e certificados
- APIs públicas expostas
- Padrões de arquitetura observáveis
- Desempenho inicial (Core Web Vitals)
- SEO e acessibilidade
- Segurança básica (CSP, HSTS, CORS, etc.)

Nota: quando houver acesso apenas à URL, a análise será limitada às informações expostas publicamente. O código-fonte interno requer integração via repositório.

6.4 Integrações Futuras com IDEs

- Visual Studio Code
- JetBrains (IntelliJ IDEA, PyCharm, WebStorm, Rider, etc.)
- Visual Studio
- Neovim
- Eclipse

6.5 APIs e Serviços

- APIs REST — análise de contratos e endpoints
- GraphQL — análise de schemas e resolvers

- gRPC — análise de protobufs
- WebSocket — análise de eventos e mensagens

CAPÍTULO 7

Funcionalidades

Conjunto completo de capacidades da plataforma, organizadas por módulo.

7.1 Leitura Inteligente

- Detecção automática de linguagens, frameworks e bibliotecas
- Identificação de ORMs, bancos de dados, ferramentas de build
- Reconhecimento de Docker, CI/CD, cloud e containers
- Detecção de package managers e configurações de projeto

7.2 Indexação e Mapeamento

- Leitura recursiva completa do projeto
- Construção de mapa de módulos e dependências
- Mapa de chamadas entre funções e serviços
- Mapa de responsabilidades por componente
- Grafo de dependências com visualização

7.3 Engenharia Reversa

- Reconstrução automática da arquitetura
- Geração de diagramas C4 (contexto, container, componente, código)
- Diagramas UML (classes, sequência, estado, atividades, componentes)
- Diagramas de banco de dados e relacionamentos
- Mapeamento de APIs e fluxos de integração

7.4 Ensino e Explicação

- Explicação do projeto completo em linguagem natural
- Explicação de pasta, arquivo, classe, interface
- Explicação de método, função, variável e linha individual
- Modo professor com analogias e exemplos didáticos
- Trilhas de aprendizado baseadas no projeto

7.5 Debug Assistido

- Compreensão do contexto antes de qualquer diagnóstico
- Localização da origem do problema
- Explicação da causa raiz
- Análise do impacto da falha
- Sugestão e comparação de soluções

7.6 Análise de Qualidade

- Detecção de código morto e duplicações
- Identificação de acoplamento excessivo e alta complexidade ciclomática
- Code Smells, violações de SOLID e Clean Code
- Padrões de DDD e arquitetura

7.7 Auditoria de Segurança

- SQL Injection, XSS, CSRF, SSRF
- Segredos, tokens e senhas expostas no código
- Dependências com vulnerabilidades conhecidas (CVEs)
- OWASP Top 10

7.8 Análise de Testes

- Identificação de testes unitários, de integração e E2E
- Estimativa de cobertura de testes
- Detecção de mocks, fixtures e frameworks utilizados
- Geração de estratégia e exemplos de testes

7.9 Análise de Performance

- Identificação de loops ineficientes e consultas lentas
- Detecção de vazamentos de memória e N+1 queries
- Análise de uso de cache e complexidade algorítmica

7.10 Documentação Automática

- Geração de README completo e atualizado
- Wiki técnica do projeto
- Documentação de endpoints e contratos de API
- Documentação de schema de banco de dados
- Geração de diagramas e fluxos em múltiplos formatos

CAPÍTULO 8

Requisitos Funcionais e Não Funcionais

8.1 Requisitos Funcionais

ID	Requisito
RF-001	Ler e indexar projetos completos de forma recursiva
RF-002	Identificar linguagens, frameworks e bibliotecas automaticamente
RF-003	Identificar e reconstruir a arquitetura do projeto
RF-004	Explicar código em qualquer nível de granularidade
RF-005	Gerar documentação automática em múltiplos formatos
RF-006	Detectar bugs, vulnerabilidades e code smells
RF-007	Sugerir melhorias sem alterar arquivos automaticamente
RF-008	Funcionar como ferramenta de ensino interativo
RF-009	Suportar múltiplas fontes de entrada de código
RF-010	Gerar diagramas e visualizações automáticas

8.2 Requisitos Não Funcionais

- Performance — respostas rápidas mesmo em projetos com milhares de arquivos
- Escalabilidade — suporte a múltiplos usuários e projetos simultâneos
- Segurança — código-fonte do usuário isolado e protegido por padrão
- Modularidade — novos analisadores e agentes podem ser adicionados sem alterar o núcleo
- Extensibilidade — suporte a novas linguagens e frameworks via plugins
- Observabilidade — logs, métricas e rastreamento em todas as operações críticas
- Disponibilidade — objetivo de 99.9% de uptime (SLA Enterprise)
- Multilíngue — interface e análises disponíveis em múltiplos idiomas
- Offline (futuro) — versão desktop com processamento local
- Privacidade por padrão — nenhum dado do usuário compartilhado sem consentimento explícito

8.3 Fora do Escopo — V1

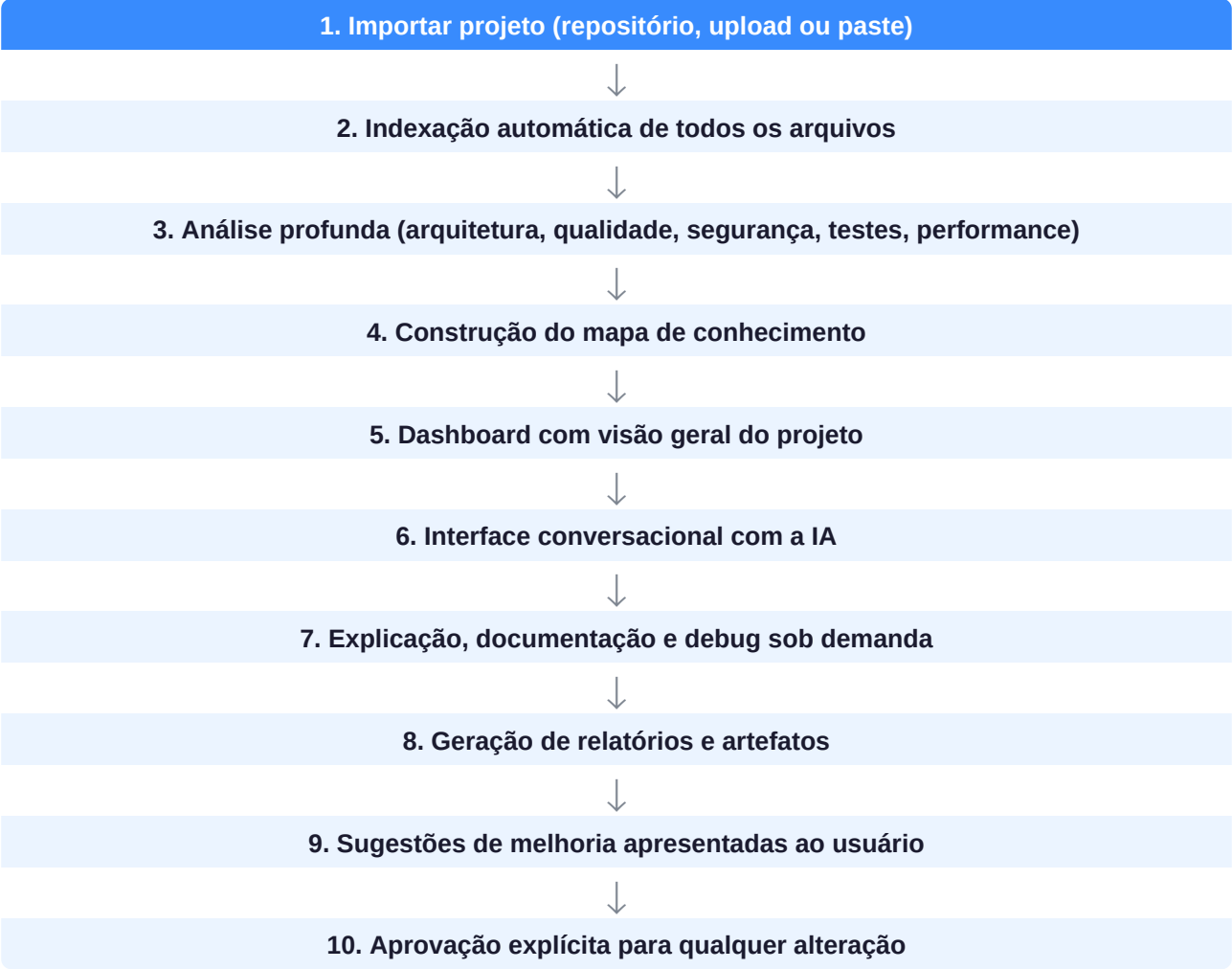
As funcionalidades abaixo estão **explicitamente fora** do escopo da versão inicial:

- Escrita automática de código sem solicitação explícita do usuário
- Deploy automático de aplicações
- Refatoração automática sem aprovação do usuário
- Execução remota de código do projeto analisado
- Alteração direta de arquivos sem confirmação explícita

CAPÍTULO 9

Fluxo do Usuário

Jornada completa desde a importação até a entrega de valor.



CAPÍTULO 10

Roadmap

Evolução planejada da plataforma em cinco fases.

Fase 1 — MVP

- Leitura recursiva de projetos
- Indexação e identificação de linguagens e frameworks
- Explicação de arquivos e funções via IA
- Geração inicial de documentação (README e Wiki)
- Suporte a upload de arquivos e repositórios GitHub

Fase 2 — Core Features

- Diagramas automáticos (C4, UML, dependências)
- Grafo de dependências interativo
- Modo Professor com trilhas de aprendizado
- Modo Debug com análise de impacto
- Auditoria de qualidade (SOLID, Clean Code, Code Smells)

Fase 3 — Segurança e Performance

- Auditoria de segurança (OWASP Top 10, CVEs)
- Análise de performance (N+1, memory leaks, complexidade)
- Análise de cobertura de testes
- Plugins para IDEs (VS Code, JetBrains)

Fase 4 — Colaboração e Integração

- Colaboração em equipe com histórico compartilhado
- Integração completa com GitHub, GitLab e Bitbucket
- Comparação entre versões e branches
- Sugestões assistidas por IA com rastreabilidade

Fase 5 — Plataforma Completa

- Agentes especializados de terceiros (marketplace)
- Aprendizado contínuo por projeto
- Suporte a múltiplos repositórios simultâneos
- Aplicativo Desktop (Electron)
- Plataforma SaaS Enterprise completa

CAPÍTULO 11

Métricas de Sucesso (KPIs)

KPI	Definição
Tempo de indexação	Tempo médio para indexar um projeto de tamanho típico
Tempo de resposta da IA	Latência média para responder perguntas sobre o código
Precisão de detecção	Acurácia na identificação de linguagens, frameworks e dependências
Qualidade da documentação	Avaliação humana e automatizada dos artefatos gerados
Satisfação do usuário	NPS e CSAT coletados pós-análise
Redução de onboarding	Tempo médio reduzido para compreender um projeto desconhecido
Cobertura de explicações	% do projeto coberto com explicações do projeto à linha
Detecção de vulnerabilidades	Acurácia e recall na detecção de issues de segurança
Engajamento	DAU/MAU, sessões por usuário, retenção em 30 dias
Conversão freemium → pago	Taxa de upgrade de plano gratuito para pago

CAPÍTULO 12

Modelo de Monetização

Plano / Canal	Descrição
Plano Free	Projetos públicos, análise básica, limite de arquivos por análise
Plano Pro	Projetos privados, análise completa, histórico, exportação PDF
Plano Teams	Multi-usuário, colaboração, integrações Git, suporte prioritário
Plano Enterprise	SSO, SAML, SLA, on-premise, agentes customizados, SLA 99.9%
Plano Educação	Desconto institucional, turmas, modo professor, relatórios de progresso
Marketplace	Agentes e extensões de terceiros com revenue share
API Pública	Acesso programático pago por uso (pay-per-call)

Riscos

Risco	Nível e Mitigação
Privacidade do código-fonte	Alto — mitigado com isolamento por tenant, criptografia e políticas claras
Alucinações da IA	Alto — mitigado com RAG, citação de evidências e indicação de confiança
Custo de tokens de LLM	Médio — mitigado com caching, chunking inteligente e modelos locais
Suporte a linguagens exóticas	Médio — mitigado com arquitetura extensível de parsers
Competição de grandes players	Médio — mitigado com foco em nicho de compreensão e ensino
Latência em projetos grandes	Médio — mitigado com indexação incremental e processamento assíncrono

CAPÍTULO 14

Constituição do Produto

Princípios invioláveis que orientam todas as decisões de produto e engenharia do GoReadCode.

14.1 O que o GoReadCode NUNCA deve fazer

- Modificar código do usuário sem solicitação e confirmação explícita
- Inventar respostas quando não houver evidências suficientes no código analisado
- Esconder limitações da análise — toda incerteza deve ser comunicada claramente
- Expor código confidencial de um projeto para outros usuários
- Priorizar velocidade de resposta em detrimento da precisão e rastreabilidade
- Armazenar código-fonte do usuário além do período necessário para a análise, sem consentimento

14.2 O que o GoReadCode SEMPRE deve fazer

- Explicar o raciocínio quando solicitado — todo diagnóstico deve ser auditável
- Preservar a rastreabilidade entre conclusões e evidências no código
- Respeitar as políticas de privacidade definidas pelo usuário e pela organização
- Indicar o nível de confiança de cada análise gerada
- Apresentar alternativas quando múltiplas interpretações são possíveis
- Colocar o contexto antes da ação — compreender sempre antes de agir

14.3 Princípio Fundamental

A IA é um apoio à inteligência humana, não uma substituição. Decisões críticas permanecem transparentes, auditáveis e sob controle do usuário. O GoReadCode amplifica a capacidade do desenvolvedor; não a substitui.

PARTE II

ARQUITETURA DE SOFTWARE

Domínio, Stack, Módulos, Eventos e Infraestrutura

CAPÍTULO A

Princípios Arquiteturais

Toda a implementação do GoReadCode deve respeitar estes princípios.

- Domain-Driven Design (DDD) — organizado em domínios de negócio, não em tecnologias
- Clean Architecture — separação clara entre domínio, aplicação e infraestrutura
- SOLID — princípios de design aplicados em toda a base de código
- API First — contratos definidos antes da implementação
- AI First — IA como cidadã de primeira classe, não como add-on
- Event-Driven — comunicação assíncrona entre domínios via eventos
- Contexto antes da ação — nenhum módulo atua sem contexto suficiente
- Observabilidade — todas operações relevantes geram eventos, métricas e logs
- Segurança por padrão — proteção de dados e código-fonte desde a concepção
- Rastreabilidade — toda conclusão ligada às suas evidências no código

CAPÍTULO B

Modelo de Domínio (DDD)

O sistema é dividido em Bounded Contexts independentes com contratos bem definidos.

B.1 Bounded Contexts

Contexto (Coluna 1)	Contexto (Coluna 2)
Workspace Context	Learning Context
Project Context	Debug Context
Code Analysis Context	Visualization Context
Reverse Engineering Context	Collaboration Context
AI Context	Administration Context
Documentation Context	Security Context

B.2 Entidades Principais

Entidade	Campos Principais
Workspace	id, nome, proprietário, organizações, configurações, projetos, histórico
Projeto	id, nome, origem, linguagem principal, frameworks, status, arquitetura
Análise	id, projeto, data, versão, duração, status, métricas, riscos
Arquivo	caminho, hash, linguagem, tamanho, módulo, dependências, símbolos
Classe	nome, namespace, métodos, propriedades, interfaces, heranças
Método	assinatura, parâmetros, retorno, complexidade, chamadas, exceções
Dependência	origem, destino, tipo (arquivo/classe/API/biblioteca), peso
Agente IA	nome, especialidade, modelo, capacidades, prioridade, nível de confiança
Conversa	usuário, projeto, mensagens, contexto, memória, sessão
Documento	tipo (README/Wiki/PDF), projeto, versão, conteúdo, data de geração

B.3 Eventos de Domínio

- ProjetoImportado, ProjetoAtualizado, ProjetoAnalisado
- ArquivoIndexado, ArquiteturaReconstruída
- DependênciaDetectada, APIMapeada
- DiagramaGerado, DocumentaçãoAtualizada
- AnáliseConcluída, BugDetectado, SugestãoCriada
- ProjetoCompartilhado, ExportaçãoGerada

CAPÍTULO C

Stack Tecnológica

Camada	Tecnologias
Frontend	Next.js 14+, React 18, TypeScript, TailwindCSS, Shadcn UI
Backend	NestJS, TypeScript, Python (análise estática e IA)
Banco Relacional	PostgreSQL com migrações versionadas
Cache / Filas	Redis (cache, sessões, filas de processamento)
Banco Vetorial	pgvector (PostgreSQL) ou Qdrant para embeddings
IA / LLMs	Multi-provider: Anthropic, OpenAI, Groq, Ollama (local)
Análise Estática	Tree-sitter (parsing multi-linguagem), AST generators
Containerização	Docker e Docker Compose (desenvolvimento e produção)
Orquestração	Kubernetes (produção / fase Enterprise)
CI/CD	GitHub Actions, testes automatizados, deploy contínuo
Observabilidade	OpenTelemetry, logs estruturados, métricas de negócio
Autenticação	OAuth 2.0, SAML 2.0 (Enterprise), JWT

CAPÍTULO D

Arquitetura Modular

Módulos principais e suas responsabilidades.

Módulo	Responsabilidade
Ingestão de Projeto	Conectores de repositório, upload, validação e normalização
Parser & AST Engine	Parsing multi-linguagem com Tree-sitter, geração de AST
Motor de Indexação	Indexação incremental, controle de versão de artefatos
Grafo de Dependências	Construção e manutenção do grafo de relações entre entidades
Banco Vetorial	Geração e armazenamento de embeddings para RAG
Motor de IA	Orquestrador de agentes, gerenciamento de contexto e memória
Documentador	Geração de README, Wiki, diagramas e relatórios
Debugger Assistido	Diagnóstico de falhas com análise de impacto e rastreabilidade
Módulo de Ensino	Trilhas, quizzes, flashcards e modo professor interativo
Visualization Engine	Preview, diagramas, sandbox e comparação visual
Interface Conversacional	Chat com IA, histórico de sessão e memória de projeto

CAPÍTULO E

Engine de Análise e Indexação

Fluxo interno da análise desde a importação até o mapa de conhecimento.



CAPÍTULO F

Infraestrutura e DevOps

- Ambientes: desenvolvimento (Docker Compose), staging, produção (Kubernetes)
- Deploy contínuo via GitHub Actions com gates de qualidade
- Multi-tenant com isolamento de dados por organização
- Backups automatizados com retenção configurável
- Disaster Recovery com RTO < 4h e RPO < 1h (Enterprise)
- Monitoramento com alertas proativos via OpenTelemetry
- Rate limiting e proteção contra abuso por API key
- Escalabilidade horizontal dos serviços de análise

PARTE III

INTELIGÊNCIA ARTIFICIAL

Agentes, RAG, Memória, Prompt Engine e Reasoning

CAPÍTULO A

Arquitetura da IA

O sistema de IA do GoReadCode é multi-agente, com orquestrador central e especialistas independentes por domínio.

A IA do GoReadCode não é um único modelo respondendo perguntas genéricas. É um sistema orquestrado de agentes especializados, cada um com acesso ao contexto construído durante a análise do projeto. O orquestrador decide qual agente (ou combinação de agentes) é mais adequado para cada solicitação, com base no tipo de pergunta e no contexto disponível.

CAPÍTULO B

Agentes Especializados

Agente	Especialidade
Arquiteto	Analisa e explica a arquitetura do projeto, padrões e decisões de design
Professor	Explica código de forma didática, com analogias e exemplos progressivos
Debugger	Diagnostica falhas, identifica causas raiz e analisa impacto de mudanças
Documentador	Gera README, Wiki, docstrings e documentação técnica estruturada
Segurança	Detecta vulnerabilidades OWASP, segredos expostos e dependências inseguras
Performance	Identifica gargalos, N+1 queries, memory leaks e ineficiências
Qualidade	Detecta code smells, violações de SOLID, duplicações e acoplamento
Testador	Analisa cobertura existente e gera estratégia e exemplos de testes
Engenheiro Reverso	Reconstrói arquitetura, fluxos e diagramas a partir do código
Orquestrador	Seleciona e coordena agentes, gerencia contexto e memória de sessão

CAPÍTULO C

RAG e Banco Vetorial

Retrieval-Augmented Generation garante respostas precisas e rastreáveis.

O RAG (Retrieval-Augmented Generation) é o mecanismo que permite aos agentes responder sobre o código específico do usuário sem precisar enviar o projeto inteiro ao LLM a cada consulta. Funciona em três etapas:

1. Indexação — código fragmentado em chunks semânticos e convertido em embeddings



2. Recuperação — busca por similaridade vetorial dos chunks mais relevantes à pergunta



3. Geração — LLM recebe a pergunta + chunks relevantes + contexto do projeto

- Embeddings gerados com modelos de código especializados
- Banco vetorial com busca híbrida (semântica + keyword)
- Reranking dos resultados para maior precisão
- Cache de embeddings para reutilização em consultas subsequentes
- Toda resposta cita as evidências do código que embasam a conclusão

CAPÍTULO D

Memória e Contexto

Tipo de Memória	Descrição
Memória de Sessão	Histórico da conversa atual, perguntas e respostas anteriores
Memória de Projeto	Contexto acumulado do projeto: arquitetura, decisões, análises
Memória de Usuário	Preferências, nível de experiência, áreas de interesse
Contexto de Arquivo	Análise do arquivo atual e seus relacionamentos
Contexto Global	Mapa completo do projeto para referências cruzadas

CAPÍTULO E

Prompt Engine e Reasoning

- Templates de prompt por tipo de análise e por agente especializado
- Injeção dinâmica de contexto relevante (RAG + memória)
- Chain-of-Thought para diagnósticos complexos
- Indicação explícita do nível de confiança por afirmação
- Fallback gracioso quando o contexto é insuficiente
- Controle de alucinações via grounding em evidências do código
- Suporte a múltiplos provedores de LLM (Anthropic, OpenAI, Groq, Ollama)
- Seleção dinâmica de modelo por custo-benefício e tipo de tarefa

PARTE IV

VISUALIZAÇÃO INTELIGENTE

Intelligent Visualization Engine — Preview, Sandbox e Diagramas

CAPÍTULO A

Intelligent Visualization Engine

Sempre que uma informação puder ser compreendida mais rapidamente por meio de uma representação visual, o GoReadCode deverá oferecer essa visualização.

A Visualization Engine é responsável por representar visualmente tudo aquilo que puder ser melhor compreendido por meio de elementos gráficos, simulações e pré-visualizações. Seu objetivo é reduzir a necessidade de interpretar apenas código ou texto, oferecendo compreensão imediata do impacto de alterações e do comportamento da aplicação.

A.1 Objetivos da Engine

- Visualizar alterações antes da aplicação em produção
- Demonstrar impactos em tempo real
- Representar fluxos arquiteturais graficamente
- Simular interfaces de usuário
- Exibir diferenças lado a lado entre versões
- Facilitar análises arquiteturais com diagramas interativos
- Apoiar aprendizado e debugging com ilustrações contextuais

CAPÍTULO B

Preview Inteligente e Sandbox

B.1 Preview de Interface

Quando tecnicamente viável, o GoReadCode gera pré-visualização de componentes e telas sem necessidade de compilar o projeto completo.

Framework	Tipo de Preview
React / Next.js	Preview automático do componente alterado
Vue / Angular	Preview da página ou componente afetado
HTML / CSS	Renderização imediata do markup e estilos
Flutter	Simulação da tela mobile
React Native	Preview do componente nativo

B.2 Sandbox Seguro

A plataforma executa alterações em um ambiente isolado, permitindo que o usuário experimente mudanças, compare resultados e descarte alterações sem risco. Nenhum arquivo original é modificado até a confirmação explícita do usuário.

B.3 Comparação Visual

O usuário visualiza lado a lado o estado anterior e o estado após a alteração proposta, para diferentes tipos de artefatos: interfaces, componentes, páginas, diagramas, APIs, banco de dados e documentação.

CAPÍTULO C

Diagramas e Análise de Impacto

C.1 Tipos de Diagramas Gerados

Diagrama	Diagrama
C4 Nível 1 — Contexto	UML — Estados
C4 Nível 2 — Containers	UML — Atividades
C4 Nível 3 — Componentes	Grafo de Dependências
C4 Nível 4 — Código	DER (Banco de Dados)
UML — Classes	Fluxo de APIs
UML — Sequência	Arquitetura de Infraestrutura

C.2 Visualização de Impacto de Alterações

Antes de modificar qualquer arquivo, a plataforma responde visualmente:

- Quais arquivos e módulos serão afetados
- Quais telas e componentes de UI dependem da mudança
- Quais APIs e contratos poderão ser impactados
- Quais testes deverão ser revisados ou criados
- Estimativa de complexidade da mudança

C.3 Linha do Tempo de Alterações

O usuário navega pela evolução do projeto como uma timeline visual, comparando versões, releases e branches lado a lado.

PARTE V

ESPECIFICAÇÃO TÉCNICA

Banco de Dados, APIs e Segurança

CAPÍTULO A

Banco de Dados

Modelagem das principais entidades persistidas.

A.1 Princípios

- PostgreSQL como banco principal (relacional + vetorial via pgvector)
- Redis para cache de sessões, embeddings frequentes e filas
- Todas as entidades possuem UUID como identificador primário
- Todos os artefatos são versionados (created_at, updated_at, version)
- Isolamento por tenant em todas as tabelas de dados de usuário
- Soft delete preferido ao hard delete para auditoria
- Migrações versionadas e reversíveis

A.2 Tabelas Principais

Tabela	Campos Principais
workspaces	id, name, owner_id, settings, created_at, updated_at
users	id, email, name, plan, preferences, created_at
projects	id, workspace_id, name, origin, status, language, created_at
project_analyses	id, project_id, version, status, duration, metrics, created_at
files	id, project_id, path, language, hash, size, content_ref
symbols	id, file_id, type, name, signature, line_start, line_end
dependencies	id, project_id, source_id, target_id, dep_type, weight
embeddings	id, symbol_id, vector, model, created_at
conversations	id, user_id, project_id, created_at, context
messages	id, conversation_id, role, content, agent, created_at
documents	id, project_id, type, version, content, format, created_at
diagrams	id, project_id, type, data, layout, version, created_at

CAPÍTULO B

APIs

Contratos REST principais da plataforma.

B.1 Endpoints Principais

Método	Endpoint	Descrição
POST	/api/projects/import	Importa um projeto (repositório, upload ou paste)
GET	/api/projects/:id	Retorna dados e status do projeto
GET	/api/projects/:id/analysis	Retorna análise mais recente do projeto
GET	/api/projects/:id/files	Lista todos os arquivos indexados
GET	/api/projects/:id/files/:fileId	Retorna conteúdo e análise de um arquivo
POST	/api/projects/:id/chat	Envia mensagem para a IA no contexto do projeto
GET	/api/projects/:id/diagrams	Lista diagramas gerados
POST	/api/projects/:id/documents/generate	Solicita geração de documentação
GET	/api/projects/:id/security	Retorna relatório de segurança
GET	/api/projects/:id/quality	Retorna relatório de qualidade
GET	/api/projects/:id/performance	Retorna relatório de performance
POST	/api/projects/:id/explain	Solicita explicação de arquivo, função ou linha

B.2 Padrões de API

- Autenticação via Bearer Token (JWT) em todos os endpoints protegidos
- Respostas paginadas com cursor para listas grandes
- Streaming via Server-Sent Events para respostas longas da IA
- Rate limiting por API key e por plano de usuário
- Versioning via header (Accept-Version) ou path (/api/v2/...)
- Erros padronizados com código, mensagem e campo problemático
- GraphQL disponível como alternativa ao REST para consultas complexas

CAPÍTULO C

Segurança

Segurança é um requisito não funcional de primeira classe no GoReadCode.

- Isolamento total de dados por tenant — nenhum dado de um projeto vaza para outro
- Criptografia em trânsito (TLS 1.3) e em repouso (AES-256)
- API keys nunca armazenadas em texto plano — apenas hashes
- Tokens de repositório (GitHub, GitLab) criptografados com chave de tenant
- OWASP Top 10 aplicado à própria plataforma
- Logs de auditoria imutáveis para todas as operações sensíveis
- CSP, HSTS, X-Frame-Options e demais security headers configurados
- Varredura automática de dependências da plataforma (Dependabot)
- Pentesting periódico e disclosure responsável de vulnerabilidades

UX / UI

PARTE VI

UX / UI

Identidade Visual, Telas e Acessibilidade



UX / UI

CAPÍTULO A

Identidade Visual

A.1 Paleta de Cores

Token	Hex	Uso
--accent	#388BFD	Ações primárias, links, bordas de destaque
--accent-2	#58A6FF	Versão mais clara do accent para hover
--bg	#0D1117	Background principal da aplicação
--sidebar	#161B22	Background de sidebars e painéis secundários
--panel	#1C2128	Background de painéis e modais
--border	#30363D	Bordas, divisórias, separadores
--text	#E6EDF3	Texto principal
--muted	#7D8590	Texto secundário, placeholders, rótulos
--green	#3FB950	Sucesso, status positivo, testes passando
--orange	#F0883E	Aviso, atenção, itens pendentes
--red	#FF7B72	Erro, falha, código problemático, keywords

A.2 Tipografia

Família	Uso
Interface (UI)	Inter — sans-serif moderno para textos de interface
Código	JetBrains Mono / Fira Code — mono espaçado para código
Tamanhos	10px (micro), 12px (caption), 14px (body), 16px (sub), 20px (h2), 28px (h1)

CAPÍTULO B

Telas Principais

Tela	Conteúdo Principal
Landing Page	Hero com tagline, demonstração animada, CTAs, planos e depoimentos
Onboarding	Fluxo de configuração do LLM, importação do primeiro projeto, tour guiado
Dashboard	Lista de projetos, status de análise, atividade recente, atalhos
Workspace	Sidebar com árvore de arquivos, viewer de código central, painel de análise
Chat IA	Interface conversacional com histórico, sugestões de perguntas, contexto ativo
Visualizador de Código	Código com syntax highlight, linhas clicáveis, busca, minimap
Diagramas	Visualização interativa de diagramas C4, UML, dependências e banco
Relatório de Segurança	Lista de vulnerabilidades com severidade, evidências e sugestões
Relatório de Qualidade	Code smells, métricas, acoplamento, complexidade ciclomática
Modo Professor	Explicação progressiva com blocos didáticos, quizzes e trilha
Modo Debug	Diagnóstico com causa raiz, impacto, comparação de soluções
Documentação Gerada	Preview do README/Wiki, opções de exportação (MD, PDF, HTML)
Configurações	Perfil, LLM provider, API keys, preferências, plano
Admin Enterprise	Gestão de usuários, times, projetos, uso e faturamento

CAPÍTULO C

Acessibilidade e Responsividade

- WCAG 2.1 nível AA como padrão mínimo de acessibilidade
- Suporte completo a navegação por teclado
- Screen readers (ARIA labels e roles em todos os componentes)
- Contraste mínimo de 4.5:1 para texto normal
- Layout responsivo: desktop (prioridade), tablet e mobile (consulta)
- Temas claro e escuro (dark mode como padrão)
- Redução de movimento respeitada (prefers-reduced-motion)
- Internacionalização (i18n) preparada desde o início

PARTE VII

MANUAIS E ANEXOS

Comparativo Competitivo, Glossário e Visão de Longo Prazo

CAPÍTULO A

Comparativo Competitivo

Funcionalidade	GoReadCode	Copilot	Cursor	Sourcegraph	SonarQube
Análise arquitetural	✓ Completa	✗	Parcial	Parcial	✗
Explicação linha a linha	✓	Parcial	✓	✗	✗
Modo Professor / Ensino	✓	✗	✗	✗	✗
Engenharia Reversa	✓	✗	✗	Parcial	✗
Auditoria de Segurança	✓	✗	✗	✗	✓
Geração de Documentação	✓ Completa	Parcial	Parcial	✗	✗
Multi-LLM	✓	✗	✓	✗	✗
Suporte a múltiplas fontes	✓	✗	✗	✓	✗
Diagramas automáticos	✓	✗	✗	Parcial	✗
Análise de Performance	✓	✗	✗	✗	Parcial
Sandbox Visual	✓	✗	✗	✗	✗

CAPÍTULO B

Glossário

Termo	Definição
AST	Abstract Syntax Tree — representação estruturada do código após parsing
Bounded Context	Limite explícito de um domínio no DDD, com modelo próprio
Code Intelligence	Conjunto de capacidades de análise semântica de código-fonte por IA
Code Smell	Padrão no código que indica possível problema de design
Embedding	Representação vetorial numérica de texto ou código para busca semântica
Grafo de Dependências	Estrutura que mapeia as relações entre módulos, arquivos e funções
LLM	Large Language Model — modelo de linguagem de grande escala
MCP	Model Context Protocol — protocolo para comunicação entre agentes e ferramentas
N+1 Query	Anti-padrão onde N consultas extras são geradas para cada item de uma lista
OWASP Top 10	Lista das 10 vulnerabilidades mais críticas em aplicações web
Parser	Componente que lê código-fonte e o transforma em estrutura analisável
RAG	Retrieval-Augmented Generation — geração aumentada por recuperação de contexto
Tenant	Cliente ou organização isolada dentro de uma plataforma multi-inquilino
Tree-sitter	Parser multi-linguagem incremental usado para análise de código
Vector DB	Banco de dados otimizado para armazenar e consultar vetores de embedding

CAPÍTULO C

Visão de Longo Prazo (5–10 anos)

Estratégia de evolução e novas fronteiras do GoReadCode.

C.1 Evolução da Plataforma

- Análise contínua de software com detecção proativa de regressões
- Preservação e transferência de conhecimento organizacional sobre sistemas
- Marketplace de agentes especializados desenvolvidos por terceiros
- Colaboração em tempo real entre múltiplos desenvolvedores no mesmo projeto
- Suporte a paradigmas emergentes (computação quântica, edge computing, WebAssembly)
- Integração com ecossistemas corporativos (Jira, Confluence, ServiceNow, SAP)
- Agentes com memória de longo prazo e aprendizado contínuo por organização

C.2 Áreas de Expansão

- GoReadCode for Education — plataforma dedicada ao ensino superior e técnico
- GoReadCode Enterprise On-Premise — versão completamente local para setores regulados
- GoReadCode CLI — ferramenta de linha de comando para integração em pipelines CI/CD
- GoReadCode Mobile — versão para revisão de código e consulta em dispositivos móveis
- GoReadCode for Open Source — tier gratuito expandido para projetos open source

C.3 Tendências Incorporadas

- Modelos de IA especializados em código com raciocínio (reasoning models)
- Análise multimodal — diagramas, wireframes e código em conjunto
- Agentes autônomos com supervisão humana para tarefas repetitivas de qualidade
- Plataforma de Code Intelligence como standard na engenharia de software